

Explorando los LLM por dentro

Guion completo, palabra por palabra, para una masterclass de 60 minutos

Basado en la PPT "Masterclass LLM por Dentro" y en la estructura del repositorio de la clase. El objetivo no es añadir teoría innecesaria, sino darte exactamente qué explicar, qué mostrar y qué hacer para que la sesión sea sólida y controlable.

Regla de oro: no intentes explicar todo lo que existe sobre LLMs. Sigue la historia: texto → tokens → embeddings → Transformer → attention → siguiente token → familias de modelos → decisión de ingeniería → RAG.

Duración objetivo: 60 minutos. El guion está escrito para leerse con naturalidad; puedes recortar ejemplos si la clase participa mucho.

1. Mapa de la clase y tiempos

Tiempo	Bloque	Objetivo
0-5	Hook	Romper la idea de "mejor LLM".
5-10	Historia y Conceptos	Evolución tecnológica, redes neuronales y modelos fundacionales.
10-25	Anatomía	Entender tokens → embeddings → Transformer → atención → generación.
25-32	Modelos	Situar GPT, Gemini, Claude, Llama, Mistral y DeepSeek.
32-40	Demos	Ver tokens y Transformer.
40-49	Ejercicio	Pensar como ingeniero.
49-52	Puesta en común	Extraer requisitos/criterios/evidencia.
52-57	RAG	Entender cómo se añade conocimiento externo.
57-60	Cierre	Consolidar tres ideas.

La presentación sigue la columna vertebral: entrada de texto → tokens → embeddings; proceso de contexto y attention en Transformer; salida autoregresiva; y finalmente ecosistema y selección de ingeniería.

2. DIAPOSITIVA 1 - Explorando los LLM por dentro

Tiempo: 1 minuto | Qué ves: título, subtítulo y la secuencia Texto → Tokens → Vectores → Contexto → Próximo Token.

"Bienvenidos. Hoy vamos a intentar responder una pregunta muy sencilla: ¿qué ocurre realmente cuando escribimos una frase a un modelo de lenguaje? No quiero que esta hora se convierta en una lista de nombres de modelos. Quiero que entendamos primero qué está haciendo un LLM por dentro y, después, cómo utilizar ese conocimiento para tomar decisiones de ingeniería."

"Durante la sesión vamos a seguir una historia muy concreta. Partimos del texto que escribimos. Ese texto se transforma en tokens. Los tokens se convierten en representaciones numéricas. Después un Transformer usa el contexto para transformar esas representaciones y, finalmente, el modelo produce probabilidades sobre cuál debe ser el siguiente token."

"Al final utilizaremos esta base para comparar GPT, Gemini, Claude, Llama, Mistral y DeepSeek, y para entender por qué en un sistema real un LLM normalmente es solo una pieza más. Y terminaremos viendo RAG."

**Pregunta para la clase: "Cuando vosotros escribís algo en ChatGPT, ¿qué creéis que recibe realmente el modelo?"*

(Transición: no respondas todavía; pasa a la siguiente slide).

3. DIAPOSITIVA 2 - ¿Cuál es el mejor LLM?

Tiempo: 4 minutos

"Quiero empezar con una pregunta provocadora: ¿cuál es el mejor LLM?"

"Decidme uno. GPT. Gemini. Claude. Llama. Mistral. DeepSeek. Lo que queráis. Ahora os hago una segunda pregunta: ¿para qué?"

"Y aquí está la idea central de toda la sesión. Sin conocer el problema que queremos resolver, decir que un modelo es 'el mejor' no es una afirmación técnicamente completa. Para decidir necesitamos saber qué precisión necesitamos, cuánto podemos gastar, qué latencia aceptamos, si los datos son sensibles, cuánto control necesitamos y dónde vamos a ejecutar el sistema."

"Por eso hoy no vamos a intentar construir un ranking definitivo. Vamos a aprender una forma de pensar. Primero entenderemos cómo funciona un LLM. Después veremos diferentes familias de modelos. Y al final utilizaremos requisitos, restricciones y evidencia para decidir."

"Quiero que os quedéis ya con una primera frase: no elegimos el modelo primero. Definimos el problema primero."

(Apunta en pantalla: Elección = Requisitos + Restricciones + Evidencia. No la desarrolles aún: volverás a ella después).

4. DIAPOSITIVA 3 (NUEVA) - Cómo llegamos hasta aquí

Tiempo: 5 minutos

"Antes de entrar en cómo funciona un LLM por dentro, quiero que entendamos cómo hemos llegado hasta aquí."

"La idea de predecir lenguaje es mucho anterior a ChatGPT. Primero tuvimos modelos de lenguaje más simples y después pasamos a usar redes neuronales para intentar capturar la secuencia del texto."

"Hagamos un brevísimo paréntesis: ¿qué es una red neuronal? En el fondo, es un sistema organizado en capas que procesa información realizando cálculos matemáticos. La información avanza en una dirección a través de lo que llamamos 'forward pass', pasando por funciones de activación. La red toma decisiones basándose en lo que llamamos 'parámetros', que no son más que pesos (weights) y sesgos (biases)."

"¿Cómo aprende esos pesos? Mediante el 'entrenamiento' (training). La red procesa un dataset inmenso, evalúa su error o 'loss', y utiliza un proceso matemático llamado backpropagation y descenso del gradiente para ajustar esos pesos paso a paso (step) en pequeños bloques de datos (batch), repitiendo el proceso en varias pasadas completas llamadas épocas (epochs). Esa fase es muy costosa. Sin embargo, cuando nosotros usamos hoy un modelo, no lo estamos entrenando. Estamos haciendo 'inferencia': simplemente usamos esos parámetros ya aprendidos para generar una respuesta."

"Sabiendo esto, volvamos a la historia. Durante años, para procesar texto, se utilizaron arquitecturas de redes recurrentes, como RNN y LSTM. Pero tenían un problema fundamental: procesaban palabra por palabra y tenían dificultades enormes para trabajar eficientemente con secuencias largas."

"En 2017 aparece el Transformer, una arquitectura que cambia radicalmente la forma de procesar secuencias porque permite utilizar mecanismos de 'atención' para relacionar diferentes posiciones del texto al mismo tiempo."

"A partir de ahí se empieza a escalar el entrenamiento: usamos muchísimos más datos, muchísimos más parámetros y mucha más capacidad de cómputo. Aparecen entonces los llamados **modelos fundacionales**. ¿Qué es un modelo fundacional? Es un modelo entrenado a una escala gigantesca sobre datos generales, que después puede adaptarse o utilizarse como base para muchas tareas distintas."

"Las familias como GPT o BERT nacen como modelos fundacionales especializados en lenguaje (LLMs). Después, estos modelos dejan de ser únicamente sistemas que completan texto prediciendo la siguiente palabra. Empiezan a someterse a procesos de instruction tuning y alineamiento (RLHF) para aprender a seguir instrucciones y mantener conversaciones reales con nosotros."

"Y hoy un modelo de lenguaje ya no se utiliza necesariamente solo. Podemos combinarlo con herramientas, recuperación de información externa (RAG), documentos propios, o incluso darle capacidades multimodales para que vea imágenes."

"Por eso hoy no solo nos interesa saber qué modelo existe. Nos interesa entender cómo funciona y cómo construir sistemas alrededor de él."

(Transición directa): "Entonces, ¿qué ocurre realmente cuando escribimos una frase a un LLM?"

5. DIAPOSITIVA 4 - Tokenización

Tiempo: 2-3 minutos + demo después

"Primera sorpresa: un LLM no recibe una frase como nosotros la vemos. Nosotros vemos palabras y caracteres. El modelo recibe una secuencia de unidades llamadas tokens. Como veis en la diapositiva, el LLM no procesa caracteres ni palabras completas."

"Un tokenizer toma el texto y lo divide en unidades que pueden ser palabras completas, partes de palabras, signos o fragmentos frecuentes. Cada una de esas unidades queda representada mediante un identificador numérico."

"Así que una frase como 'Estoy aprendiendo inteligencia artificial' no se convierte necesariamente en cuatro piezas porque tenga cuatro palabras. Podría dividirse de otra manera. Y esa forma de segmentar depende del tokenizer que estemos usando."

"Esto importa porque el número de tokens no es una curiosidad. Afecta a cuánto contenido cabe en el contexto y, en muchos servicios comerciales, tiene una relación directa con el coste de computación."

(Frase de seguridad: "No digáis que un token es una palabra. Una palabra puede convertirse en uno o varios tokens, y un token tampoco tiene por qué coincidir exactamente con una palabra").

6. DEMO - Tiktokenizer: exactamente qué hacer

Tiempo total: 3-4 minutos | URL: <https://tiktokenizer.vercel.app/>

- **Paso 1:** Déjalo en el tokenizer que tengas seleccionado.
- **Paso 2:** Empieza con una frase corta. Antes de mirar el contador pregunta: "¿Cuántos tokens creéis que tiene?".
- Prueba A: *I am learning artificial intelligence.*
- Prueba B: *Estoy aprendiendo inteligencia artificial.*
- **(Importante: si ambas tienen el mismo número, NO pasa nada. La conclusión correcta es que el número de tokens depende de la segmentación concreta del tokenizer).**
- Prueba C: usa código (ej: una función corta en Python).
- Prueba D: un emoji (❤️).

"Lo que quiero que observéis es que el modelo no trabaja con 'palabras' como unidad fundamental. Trabaja con tokens. Una misma cantidad de palabras puede terminar representada mediante un número diferente de tokens dependiendo del texto y del tokenizer."

7. DIAPOSITIVA 5 - De tokens a representaciones: embeddings

Tiempo: 3 minutos

"Ahora tenemos tokens e IDs numéricos. Pero un número como 5832 no contiene significado por sí mismo. Es solamente un identificador discreto y sin correlación. Para que una red neuronal trabaje con esa información necesitamos convertir el token en una representación numérica continua: un embedding."

"Un embedding es un vector. En vez de pensar que el token es simplemente el número 5832, pensamos que ese token está asociado a una lista de valores o coordenadas matemáticas, por ejemplo [0.2, -0.7, 0.4...]. La red trabaja con esas representaciones vectoriales."

"Podemos imaginar que todos esos vectores viven en un espacio de muchas dimensiones. La geometría de ese espacio puede capturar relaciones útiles entre palabras o conceptos (cercanía geométrica equivale a cercanía semántica). Pero hay una precisión importante: el embedding inicial no es todavía una representación completamente contextualizada."

"Por ejemplo, la palabra 'banco' puede aparecer en 'el banco aprobó el préstamo' o en 'me senté en el banco del parque'. El mismo token inicial no sabe si es una entidad financiera o un asiento. Esa información contextual se construye a medida que la representación atraviesa el Transformer."

8. DIAPOSITIVA 6 - El bloque Transformer

Tiempo: 4 minutos

"Ahora necesitamos un mecanismo que permita que estas representaciones incorporen contexto. Ese mecanismo es el Transformer."

"El Transformer refina las representaciones vectoriales capa a capa, mediante una serie de bloques que se repiten. En términos simples, cada bloque combina mecanismos para mezclar información entre posiciones y para transformar esa información."

"La primera gran pieza es Self-Attention. Su función es mezclar información, permitiendo que la representación de un token tenga en cuenta información de otros tokens de la secuencia."

"La segunda gran pieza es la red feed-forward o MLP. Mientras la atención mezcla información entre tokens, el MLP procesa y transforma la representación individual de cada posición mediante capas lineales."

"Además, los bloques utilizan conexiones residuales y normalización de capa (LayerNorm). Las conexiones residuales permiten conservar la representación anterior y sumarle una actualización. LayerNorm ayuda a mantener los valores estables para facilitar el flujo por la red."

9. DIAPOSITIVA 7 - Self-Attention

Tiempo: 4-5 minutos + BBycroft

"Esta es una de las ideas más importantes de la clase. Self-Attention permite que cada token decida de qué otros tokens necesita información para construir su contexto."

"Volvamos al ejemplo: 'The bank approved the loan'. Si procesamos 'bank', palabras como 'approved' y 'loan' son especialmente relevantes y nos conectan con el mundo de las finanzas. Si cambiamos la frase a: 'The bank is next to the river', ahora 'river' aporta contexto de geografía y agua. La atención permite que el vector de 'bank' absorba esa información del entorno."

"Para conseguirlo, cada token genera tres vectores: Query, Key y Value.

- **Query** es lo que estoy buscando.
- **Key** es qué tipo de información ofrezco.
- **Value** es la información real que puedo comunicar o aportar."

"El Query de un token se compara matemáticamente contra las Keys de los demás. De ahí salen unas puntuaciones o scores de atención. Una puntuación alta significa que esa posición es muy relevante. Después usamos esos pesos para combinar y sumar los Values."

"En un modelo que genera texto (Transformer causal) hay una regla de oro: un token no puede mirar hacia el futuro. Cuando generamos de izquierda a derecha, usamos una máscara causal para tapar las posiciones futuras."

10. DEMO - BBycroft: cómo usarla sin perderte

Tiempo: 4-5 minutos | URL: <https://bbycroft.net/llm>

(Selecciona GPT-2 (small) si está disponible. Recorrido exacto: Embedding → Self-Attention → Q/K/V → causal masking → softmax → MLP → repetición de bloques → output).

"Esta herramienta interactiva en 3D nos permite hacer visible lo que acabamos de explicar. No quiero recorrer cada multiplicación; quiero seguir el camino de una representación."

"Primero, vemos el embedding. A partir de aquí la red trabaja con vectores, no con palabras."

"Ahora voy a Self-Attention. Cada token produce Query, Key y Value. La Query se compara con las Keys y de ahí salen los pesos de atención. Fijaos en que estos pesos no son la respuesta final, son la forma de decidir cuánta información tomar de los demás para combinar los Values."

"Aquí también vemos la máscara causal: durante la generación un token no puede utilizar información del futuro."

"Llegamos al MLP. La atención mezcló información; ahora el MLP transforma la representación individual de cada posición."

"Todo esto forma un bloque Transformer y estos bloques se repiten. En la salida, obtenemos una distribución de probabilidades sobre los posibles siguientes tokens."

11. DIAPOSITIVA 8 - Generación token a token

Tiempo: 2 minutos

"Ya hemos visto cómo se construye el contexto. ¿Cómo produce el modelo la respuesta?"

"Un LLM no produce toda la respuesta de golpe. Opera en un bucle autoregresivo: realiza una predicción probabilística sobre el siguiente token único, lo genera, y lo reinyecta inmediatamente al contexto para repetir el proceso."

"Si el contexto termina en 'The capital of France is', el modelo asigna probabilidades. *Paris* tendría una probabilidad altísima. Seleccionamos ese token, se añade a la entrada, y volvemos a calcular."

"Aquí es donde ajustamos los hiperparámetros de sampling:

- **Temperature:** Controla el aplanamiento de la distribución. No es estrictamente 'creatividad', sino que suaviza o concentra las probabilidades, afectando el determinismo de la salida.
- **Top-k y Top-p:** Recortan la cola inferior limitando el muestreo a los candidatos más probables."

12. DIAPOSITIVA 9 - El ecosistema: GPT, Gemini, Claude, Llama, Mistral y DeepSeek

Tiempo: 7 minutos

"Volvemos a la pregunta inicial: ¿qué modelos existen? Nuestra clase abarca seis familias: GPT, Gemini, Claude, Llama, Mistral y DeepSeek."

"No penséis que son seis tecnologías completamente distintas. Comparten el mismo fundamento base, pero representan distintas decisiones de arquitectura, entrenamiento, licencias y despliegue."

"**GPT, Gemini y Claude** son familias comerciales de APIs gestionadas (OpenAI, Google, Anthropic). Su ventaja es la mínima fricción operativa y un altísimo rendimiento en benchmarks. El coste escala por token y dependes de un proveedor."

"**Llama (Meta), Mistral y DeepSeek** representan opciones del ecosistema open-weight. Los pesos están disponibles y te permiten esquemas de despliegue propio. Esto aporta privacidad total, control de infraestructura y opciones de fine-tuning, pero a cambio asumes la responsabilidad operativa (ceroOps vs GPU propias)."

"Un apunte vital: open-weight y open source no son sinónimos. Que los pesos del modelo estén disponibles no significa que todo el código, los datos y las licencias cumplan estrictamente la definición histórica de open source."

13. DIAPOSITIVA 10 - Selección: requisitos, restricciones y evidencia

Tiempo: 2 minutos

"Conclusión del ecosistema: No existe 'el mejor modelo' sin contexto. Elegir un modelo es un trade-off o compromiso de arquitectura."

1. **Requisitos:** ¿Qué debe hacer? (Responder documentos, escribir código, español).
2. **Restricciones:** ¿Qué no aceptamos? (Presupuesto máximo, latencia límite, privacidad extrema).
3. **Evidencia:** ¿Cómo probamos que funciona? (Tests reales, coste por consulta)."

"La ecuación es simple: Elección = Requisitos + Restricciones + Evidencia."

14. DIAPOSITIVA 11 - La teoría se vuelve observable

Tiempo: 1 minuto para encuadrar.

"Hasta ahora hemos explicado conceptos. Ahora quiero que los veamos. Hemos utilizado Tiktokenizer para ver la entrada, y BBycroft para ver las capas."

15. DIAPOSITIVA 12 - LLM Detective: cómo lanzar el ejercicio

Tiempo: 12 minutos de trabajo + 3 minutos de puesta en común

"Ahora os toca a vosotros trabajar como ingenieros. Pensad como ingenieros utilizando este método: 1. Hipótesis, 2. Experimento, 3. Observación, 4. Conclusión."

"Parte 1: Usad Tiktokenizer para comprobar el coste real del idioma. ¿Cuántos tokens cuesta la misma frase en inglés vs español vs JSON?"

"Parte 2 Caso Real: Un bot médico interno. Tenéis información sensible, presupuesto limitado, y requerís baja latencia. ¿Qué 3 criterios justificarían descartar una API gestionada comercial para este caso de uso? No elijáis modelo todavía. Definid requisitos, restricciones, criterios y qué experimento haríais para medirlo."

(Puesta en común: Una respuesta fuerte no dice "gana Llama", dice "elegiremos en base a evidencia de latencia, coste y privacidad bajo infraestructura propia").

16. DIAPOSITIVA 13 - Un LLM no es una aplicación: RAG

Tiempo: 5 minutos

"Para terminar, el modelo (el LLM) es el motor de inferencia, pero no es toda la aplicación. Por sí solo, no tiene acceso a los documentos privados de vuestra empresa."

"Aquí entra RAG: Retrieval-Augmented Generation. El flujo es el siguiente: Primero recuperamos evidencia relevante de una base de datos documental. Después inyectamos esa información como contexto adicional en el prompt. Finalmente, el LLM genera una respuesta fundamentada en esa evidencia aportada."

"RAG no reentrena el modelo. No altera sus parámetros. Simplemente le da un documento fresco en el momento de leer la pregunta."

(Menciona rápidamente la diapositiva que muestra el RAG como una capa de contexto separada del Motor LLM y de la UI/Guardrails).

17. DEMO - RAG del repositorio

Tiempo: 3-5 minutos

(Usa el script de indexación local del repositorio y haz una pregunta como: "¿Qué significa open-weight y cómo difiere de open source?").

"En vez de preguntar a un modelo genérico de Internet, estamos buscando primero la evidencia dentro del material de nuestro curso. El sistema recupera los fragmentos y el LLM los usa."

"Fijaos en el grounding: si el sistema responde, nos muestra de dónde sacó la información. Y si le pregunto algo fuera del corpus, un buen RAG debería indicar que no tiene evidencia suficiente, en vez de inventarlo."

18. TensorFlow Projector (Opcional)

Tiempo: 0-2 minutos, solo si sobra tiempo | URL: <https://projector.tensorflow.org/>

"Si tuviéramos que explorar estos espacios vectoriales de alta dimensión (cientos o miles de dimensiones), herramientas como PCA o t-SNE nos permiten proyectarlos a 2D o 3D para entender visualmente la relación entre conceptos."

19. FAQ DEL PROFESOR - Respuestas rápidas

- **¿Qué es un parámetro / peso (weight) / bias?** Valores numéricos ajustados durante el entrenamiento que determinan cómo la red transforma matemáticamente la entrada para hacer predicciones.
- **¿Qué es una época (epoch) y loss?** Una época es una pasada completa sobre el dataset de entrenamiento. La red evalúa su error (loss) y usa algoritmos (como backpropagation) para ir ajustando los pesos y reducir ese error.
- **¿Qué diferencia hay entre training e inference?** Training es el costoso proceso de aprender los pesos usando datos. Inferencia es lo que hacemos en ChatGPT: usar los pesos ya estáticos para procesar una entrada nueva y generar salida.
- **¿Qué es un token?** Una unidad discreta del texto representada numéricamente.
- **¿Qué es un embedding?** Una representación vectorial continua que captura cercanía semántica.
- **¿Qué hace Self-Attention?** Permite que un token incorpore información del contexto asignando pesos a otras posiciones.
- **¿Qué son Q/K/V?** Query (qué busco), Key (qué ofrezco), Value (qué información comunico).
- **¿RAG entrena al modelo?** No. Modifica dinámicamente el contexto en tiempo de consulta.

- **¿Open-weight es open source?** No necesariamente. Depende de las licencias y de qué componentes (código, datos) se liberen.

20. DIAPOSITIVA 14 - Cierre palabra por palabra

Tiempo: 3 minutos

"Quiero cerrar con tres ideas.

La primera: un LLM no procesa directamente letras como las vemos nosotros. El texto se convierte en tokens y esos tokens se convierten en representaciones vectoriales.

La segunda: el Transformer utiliza el contexto. Self-Attention permite que unas posiciones incorporen información de otras. Después, el sistema genera el siguiente token de forma autoregresiva.

La tercera: no existe un modelo ganador absoluto. Elegimos entre opciones gestionadas u open-weight basándonos en requisitos, restricciones operativas y evidencia."

"Además, recordad que un modelo aislado no es un producto final. Necesitamos herramientas como RAG, guardrails y bases de datos para integrarlo en el mundo real."

"Os dejo con una pregunta final de reflexión: si mañana tuvierais que decidir entre dos modelos para un proyecto real de vuestra empresa, ¿qué mediríais para tomar la decisión?"

(Silencio. Deja que respondan brevemente. Cierra con:)

"Exactamente. Lo importante no es tener un modelo favorito; es saber qué evidencia de ingeniería necesitamos para tomar una buena decisión. Muchas gracias."

21. CHECKLIST DE MAÑANA - Antes de entrar

- Abrir la PPT y dejar la presentación lista en la diapositiva 1.
- Abrir Tiktokenizer en otra pestaña y probar las frases hoy mismo.
- Abrir BBycroft, seleccionar GPT-2 (small) y practicar la ruta (Embedding → Q/K/V → MLP).
- Probar el comando de indexación del RAG localmente en el repositorio.
- **Control de tiempos:** Si se alarga la historia inicial o la discusión, recorta el TensorFlow Projector (opcional) pero no sacrifiques Tokens → Transformer → Selección.

- *Frase salvavidas:* "Volvamos al flujo: texto → tokens → representaciones → Transformer → siguiente token. ¿En qué parte estamos ahora?"